

IsaacLab 프로젝트를 처음부터 생성하기

가이드 웹사이트:

<https://isaac-sim.github.io/IsaacLab/main/source/overview/own-project/template.html>

1. conda 에서 isaacrab 환경에 들어간 뒤, IsaacLab 폴더로 이동

```
conda activate env_isaacrab
cd IsaacLab/
```

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ conda activate env_isaacrab
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ cd ~
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~$ cd IsaacLab
bash: cd: IsaacLab: No such file or directory
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~$ cd IsaacLab/
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ^C
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$
```

2. 새로운 프로젝트 생성 (external / internal 여부 선택)

```
./isaacrab.sh --new
```

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ./isaacrab.sh --new
[INFO] Installing template dependencies...

[INFO] Running template generator...

? Task type:
> External
  Internal
```

- Enter 키: 결정
- 방향키 ↑ ↓ : 옵션 선택

3. 프로젝트를 생성할 위치 결정

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ./isaacrab.sh --new
[INFO] Installing template dependencies...

[INFO] Running template generator...

? Task type: External
? Project path: /home/user/
```

4. 프로젝트 이름 결정

```
(env_isaacrlab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ./isaacrlab.sh --new
[INFO] Installing template dependencies...

[INFO] Running template generator...

? Task type: External
? Project path: /home/user/
? Project name: 
```

5. 사용할 에이전트 개수 선택

(지금은 single agent 선택)

```
? Task type: External
? Project path: /home/user/
? Project name: practice

RL environment features support according to Isaac Lab workflows



| Environment feature                             | Direct | Manager-based |
|-------------------------------------------------|--------|---------------|
| Single-agent                                    | yes    | yes           |
| Multi-agent                                     | yes    | no            |
| Fundamental/composite spaces (apart from 'Box') | yes    | no            |



? Isaac Lab workflow: 
> ○ Direct | single-agent
  ○ Direct | multi-agent
  ○ Manager-based | single-agent
  -----
  ○ all
```

- 스페이스바로 선택
- 선택되면 점이 초록색으로 표시됨

6. 학습에 사용할 알고리즘/프로토콜 선택

(여기서는 rsl_rl 선택)

```
Supported RL libraries

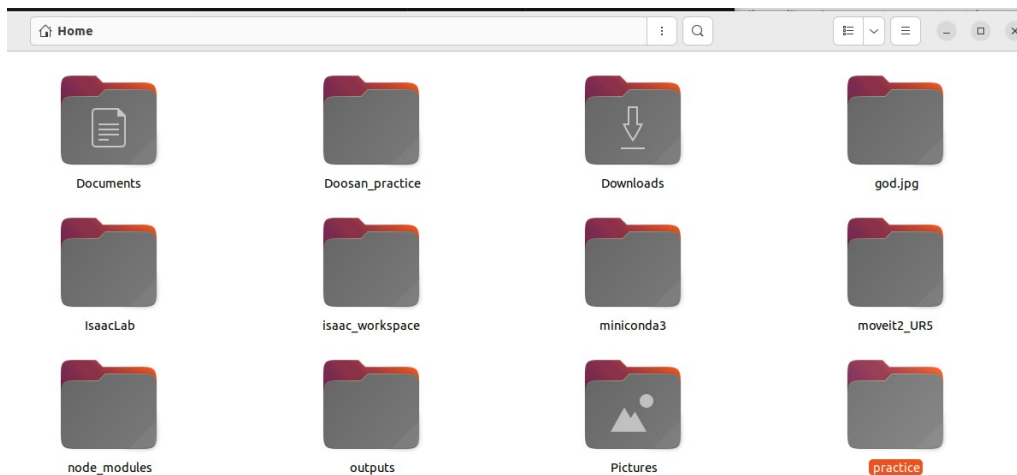


| RL/training feature          | rl_games | rsl_rl  | skrl                  | sb3     |
|------------------------------|----------|---------|-----------------------|---------|
| ML frameworks                | PyTorch  | PyTorch | PyTorch, JAX          | PyTorch |
| Relative performance         | ~1X      | ~1X     | ~1X                   | ~0.03X  |
| Algorithms                   | PPO      | PPO     | AMP, IPPO, MAPPO, PPO | PPO     |
| Multi-agent support          | no       | no      | yes                   | no      |
| Distributed training         | yes      | no      | yes                   | no      |
| Vectorized training          | yes      | yes     | yes                   | no      |
| Fundamental/composite spaces | no       | no      | yes                   | no      |



? RL library: rsl_rl
```

7. 완료되면, 앞에서 지정한 위치에 새로운 프로젝트 폴더가 생성됨



8. 터미널에서 해당 폴더로 이동

`cd ~/<folder name>`

예시 (내 경우):

`cd ~/practice`

```
(env_isaac) user@user-Vector-16-HX-AI-A2XWIG:~$ cd ~/practice
(env_isaac) user@user-Vector-16-HX-AI-A2XWIG:~/practice$
```

9. 프로젝트 설치 (editable 모드)

`python -m pip install -e source/<given-project-name>`

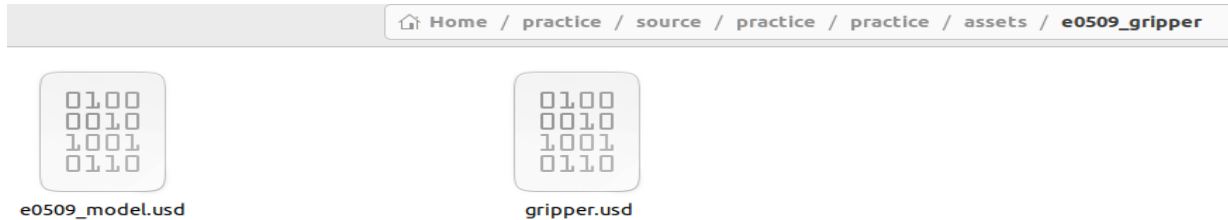
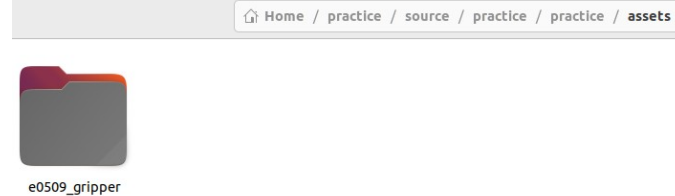
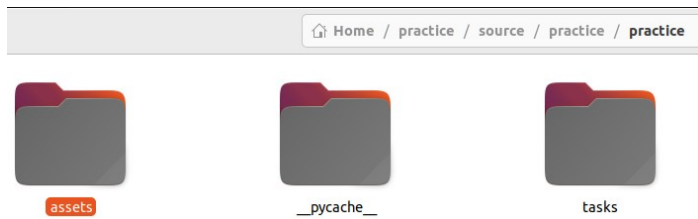
```
(env_isaac) user@user-Vector-16-HX-AI-A2XWIG:~/practice$ python -m pip install -e source/practice
Obtaining file:///home/user/practice/source/practice
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: psutil in /home/user/miniconda3/envs/env_isaac/lib/python3.11/site-packages (from practice==0.1.0) (5.9.8)
Building wheels for collected packages: practice
  Building editable for practice (pyproject.toml) ... done
  Created wheel for practice: filename=practice-0.1.0-0.editable-py3-none-any.whl size=2915 sha256=341e29aaa9b347b146b9436634d3bb5c85268a332459d61bf3da04baf38bc4af
  Stored in directory: /tmp/pip-ephem-wheel-cache-k779z10g/wheels/af/f0/36/9a1d45814eaeda261fe28e4958522e423a7e375e14592e4c5d
Successfully built practice
Installing collected packages: practice
  Attempting uninstall: practice
    Found existing installation: practice 0.1.0
    Uninstalling practice-0.1.0:
      Successfully uninstalled practice-0.1.0
  Successfully installed practice-0.1.0
```

예시:

`python -m pip install -e source/practice`

10. 아래 위치에 assets 폴더 생성

/home/user/practice/source/practice/practice



11. 아래 위치의 cfg.py 파일 열기

/home/user/practice/source/practice/practice/tasks/direct/practice/
practice_env_cfg.py

그리고 파일 전체를 아래 내용으로 교체:

```
# Copyright (c) 2022-2025, The Isaac Lab Project Developers
# (https://github.com/isaac-sim/IsaacLab/blob/main/CONTRIBUTORS.md).
# All rights reserved.
#
# SPDX-License-Identifier: BSD-3-Clause

from pathlib import Path

_PKG_ROOT = Path(__file__).resolve().parents[3] # .../source/practice/practice
DOOSAN_USD = str(_PKG_ROOT / "assets/e0509_gripper/e0509_model.usd")

import isaacsim as sim_utils
from isaacsim.actuators import ImplicitActuatorCfg

from isaacsim.assets import ArticulationCfg
from isaacsim.envs import DirectRLEnvCfg
from isaacsim.scene import InteractiveSceneCfg
from isaacsim.sim import SimulationCfg
from isaacsim.utils import configclass

@configclass
class PracticeEnvCfg(DirectRLEnvCfg):
    # env
    decimation = 2
    episode_length_s = 5.0
    # - spaces definition
    action_space = 6
    observation_space = 18
    state_space = 0
```

```

# simulation
sim: SimulationCfg = SimulationCfg(dt=1 / 120, render_interval=decimation)

# robot(s)
robot_cfg: ArticulationCfg = ArticulationCfg(
    prim_path="/World/envs/env_*/Robot",
    spawn=sim_utils.UsdFileCfg(
        usd_path=D00SAN_USD,
        activate_contact_sensors=False,
        articulation_props=sim_utils.ArticulationRootPropertiesCfg(
            enabled_self_collisions=False,
            solver_position_iteration_count=8,
            solver_velocity_iteration_count=1,
        ),
        collision_props=sim_utils.CollisionPropertiesCfg(
            collision_enabled=True,
            contact_offset=0.005,
            rest_offset=0.0,
        ),
    ),
    init_state=ArticulationCfg.InitialStateCfg(
        pos=(-0.45, 0.0, 0.53),
        joint_pos={
            "joint_1": 0.0,
            "joint_2": -1.2,
            "joint_3": 1.5,
            "joint_4": -1.8,
            "joint_5": 0.0,
            "joint_6": 1.6,
            "rh_*": 0.0,
        },
    ),
    actuators={
        "arm": ImplicitActuatorCfg(
            joint_names_expr=["joint_[1-6]"],
            stiffness=1500.0,
            damping=100.0,
        ),
        "gripper": ImplicitActuatorCfg(
            joint_names_expr=["rh_l1", "rh_r1_joint"],
            stiffness=2e4,
            damping=1e2,
        ),
    },
)

# scene
scene: InteractiveSceneCfg = InteractiveSceneCfg(num_envs=64,
env_spacing=4.0, replicate_physics=True)

# custom parameters/scales
# --- Reach task params ---
action_scale = 0.25
target_radius = 0.05
target_pos = (0.45, 0.0, 0.55)
rew_scale_dist = 1.0
rew_scale_action = 0.01
rew_scale_terminated = -2.0
rew_scale_pole_pos = -1.0
rew_scale_cart_vel = -0.01
rew_scale_pole_vel = -0.005

```

12. 아래 위치의 `practice_env.py` 파일 열기

`/home/user/practice/source/practice/practice/tasks/direct/practice`

그리고 파일 전체를 아래 내용으로 교체:

```
# Copyright (c) 2022-2025, The Isaac Lab Project Developers
# (https://github.com/isaac-sim/IsaacLab/blob/main/CONTRIBUTORS.md).
# All rights reserved.
#
# SPDX-License-Identifier: BSD-3-Clause

from __future__ import annotations

import torch

from collections.abc import Sequence

import isaacsim as sim_utils
from isaacsim.assets import Articulation
from isaacsim.envs import DirectRLEnv
from isaacsim.sim.spawners.from_files import GroundPlaneCfg, spawn_ground_plane

from .practice_env_cfg import PracticeEnvCfg

class PracticeEnv(DirectRLEnv):
    """PracticeEnv class"""
    def __init__(self, cfg: PracticeEnvCfg, render_mode: str | None = None, **kwargs):
        """__init__ method"""
        super().__init__(cfg, render_mode, **kwargs)

        # ---- Arm joint mapping (known joint names) ----
        self._arm_joint_names = [f"joint_{i}" for i in range(1, 7)]
        self._arm_dof_ids, _ = self.robot.find_joints(self._arm_joint_names)
```

```

# ---- Pick an end-effector body ONCE ----
candidates = ["tool0", "ee_link", "end_effector", "flange", "tcp"]
ee_id = None
for name in candidates:
    try:
        ids, _ = self.robot.find_bodies(name)
        if len(ids) > 0:
            ee_id = int(ids[0])
            break
    except Exception:
        pass

# Fallback: last body
self._ee_body_id = ee_id if ee_id is not None else -1

# ---- Fixed target position in "env-local" frame ----
# We convert it to world frame by adding scene.env_origins each step.
self.target_pos = torch.tensor(self.cfg.target_pos, device=self.device).repeat(self.num_envs, 1)

# Optional: helpful debug (uncomment once if you want)
# print("EE body id:", self._ee_body_id)
# try:
#     print("EE body name:", self.robot.body_names[self._ee_body_id])
# except Exception:
#     pass

def _setup_scene(self):
    # Create robot articulation from cfg
    self.robot = Articulation(self.cfg.robot_cfg)

    # Add ground plane

```

```

spawn_ground_plane(prim_path="/World/ground", cfg=GroundPlaneCfg())

# Clone and replicate environments
self.scene.clone_environments(copy_from_source=False)

# Explicitly filter collisions for CPU simulation
if self.device == "cpu":
    self.scene.filter_collisions(global_prim_paths=[])

# Add articulation to scene
self.scene.articulations["robot"] = self.robot

# Add lights
light_cfg = sim_utils.DomeLightCfg(intensity=2000.0, color=(0.75, 0.75, 0.75))
light_cfg.func("/World/Light", light_cfg)

def _pre_physics_step(self, actions: torch.Tensor) -> None:
    # Clamp actions to [-1, 1]
    self.actions = torch.clamp(actions, -1.0, 1.0)

def _apply_action(self) -> None:
    # Delta joint-position targets for the 6 arm joints
    q_arm = self.robot.data.joint_pos[:, self._arm_dof_ids] # (num_envs, 6)
    dq = self.cfg.action_scale * self.actions # (num_envs, 6)
    q_target = q_arm + dq

    # Apply joint position targets only to arm joints
    self.robot.set_joint_position_target(q_target, joint_ids=self._arm_dof_ids)

def _get_observations(self) -> dict:
    # Arm joint state
    q = self.robot.data.joint_pos[:, self._arm_dof_ids] # (num_envs, 6)

```

```

qd = self.robot.data.joint_vel[:, self._arm_dof_ids] # (num_envs, 6)

# End-effector world position
ee_pos = self.robot.data.body_pos_w[:, self._ee_body_id, :] # (num_envs, 3)

# Target world position (env origin + local target)
target_pos_w = self.scene.env_origins + self.target_pos # (num_envs, 3)

# Observation = 18 dims
obs = torch.cat([q, qd, ee_pos, target_pos_w], dim=-1) # (num_envs, 18)
return {"policy": obs}

def _get_rewards(self) -> torch.Tensor:
    ee_pos = self.robot.data.body_pos_w[:, self._ee_body_id, :]
    target_pos_w = self.scene.env_origins + self.target_pos
    dist = torch.linalg.norm(ee_pos - target_pos_w, dim=-1)

    # Dense reward: closer is better
    rew_dist = -self.cfg.rew_scale_dist * dist

    # Small action penalty
    rew_act = -self.cfg.rew_scale_action * torch.sum(self.actions**2, dim=-1)

    return rew_dist + rew_act

def _get_dones(self) -> tuple[torch.Tensor, torch.Tensor]:
    ee_pos = self.robot.data.body_pos_w[:, self._ee_body_id, :]
    target_pos_w = self.scene.env_origins + self.target_pos
    dist = torch.linalg.norm(ee_pos - target_pos_w, dim=-1)

    terminated = dist < self.cfg.target_radius
    time_out = self.episode_length_buf >= self.max_episode_length - 1

```

```
        return terminated, time_out

def _reset_idx(self, env_ids: Sequence[int] | None):
    if env_ids is None:
        env_ids = self.robot._ALL_INDICES
    super()._reset_idx(env_ids)

    # Default joint state from robot init/defaults
    joint_pos = self.robot.data.default_joint_pos[env_ids].clone()
    joint_vel = self.robot.data.default_joint_vel[env_ids].clone()

    # Place root at env origin
    default_root_state = self.robot.data.default_root_state[env_ids].clone()
    default_root_state[:, :3] += self.scene.env_origins[env_ids]

    # Write to sim
    self.robot.write_root_pose_to_sim(default_root_state[:, :7], env_ids)
    self.robot.write_root_velocity_to_sim(default_root_state[:, 7:], env_ids)
    self.robot.write_joint_state_to_sim(joint_pos, joint_vel, None, env_ids)

    # Reset target (fixed for now)
    self.target_pos[env_ids] = torch.tensor(self.cfg.target_pos, device=self.device)
```

13. 강화학습 실행

필요한 모든 파일 생성이 완료되었습니다.

아래 명령어로 강화학습 프로젝트를 실행할 수 있습니다:

```
python scripts/random_agent.py --task Template-Practice-Direct-v0 --num_envs 1
```

```
gymFactory v0.1] 100 times. Consider accessing this interface with carb::getcachedinterface() (Performance warning)
^C(env_isaac) user@user-Vector-16-HX-AI-A2XWIG:~/practice$ python scripts/random_agent.py --task Template-Practice-Direct
-v0 --num envs 1
```



Creating a IsaacLab from scratch

guide website:

<https://isaac-sim.github.io/IsaacLab/main/source/overview/own-project/template.html>

1. enter the isaacrab environment on conda and enter the isaacrab folder

```
conda activate env_isaacrab
```

```
cd IsaacLab/
```

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ conda activate env_isaacrab
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ cd ~
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~$ cd IsaacLab
bash: cd: IsaacLab: No such file or directory
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~$ cd IsaacLab/
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ^C
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$
```

2. create a new project and decide if it is external or internal

```
./isaacrab.sh --new
```

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ./isaacrab.sh --new
[INFO] Installing template dependencies...

[INFO] Running template generator...

? Task type: 
> External
  Internal
```

(press enter to decide, arrow keys to go up and down to select options)

3. decide on where to build the project:

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ./isaacrab.sh --new
[INFO] Installing template dependencies...

[INFO] Running template generator...

? Task type: External
? Project path: /home/user/
```

4. decide project name:

```
(env_isaacrab) user@user-Vector-16-HX-AI-A2XWIG:~/IsaacLab$ ./isaacrab.sh --new
[INFO] Installing template dependencies...

[INFO] Running template generator...

? Task type: External
? Project path: /home/user/
? Project name:
```

5. decide on how many agents you want (we choose single for now)

```
? Task type: External
? Project path: /home/user/
? Project name: practice

RL environment features support according to Isaac Lab workflows
```

Environment feature	Direct	Manager-based
Single-agent	yes	yes
Multi-agent	yes	no
Fundamental/composite spaces (apart from 'Box')	yes	no

```
? Isaac Lab workflow: 
> ○ Direct | single-agent
  ○ Direct | multi-agent
  ○ Manager-based | single-agent
  -----
  ○ all
```

(press space to select the option you want, the dot will light up green once selected)

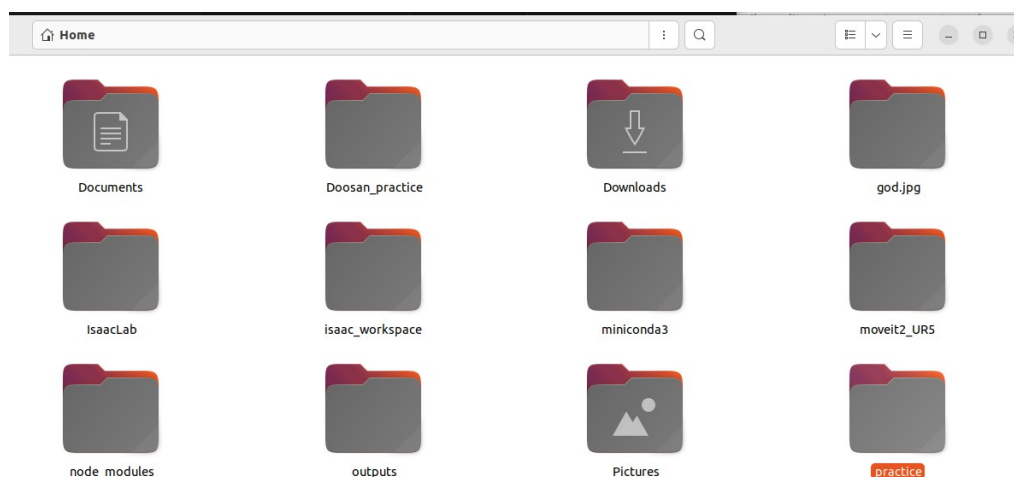
6. choose which type of algorithm/protocol we will use for training (we will choose rsl_rl)

```
Supported RL libraries
```

RL/training feature	rl_games	rsl_rl	skrl	sb3
ML frameworks	PyTorch	PyTorch	PyTorch, JAX	PyTorch
Relative performance	~1X	~1X	~1X	~0.03X
Algorithms	PPO	PPO	AMP, IPP0, MAPPO, PPO	PPO
Multi-agent support	no	no	yes	no
Distributed training	yes	no	yes	no
Vectorized training	yes	yes	yes	no
Fundamental/composite spaces	no	no	yes	no

```
? RL library: rsl_rl
```

7. Once complete a new folder (name of folder was decided earlier) will appear wherever you directed it



8. enter the folder via the terminal:

```
cd ~/<folder name>
```

or in my case:

```
cd ~/practice
```

```
(env_isaacclab) user@user-Vector-16-HX-AI-A2XWIG:~$ cd ~/practice
(env_isaacclab) user@user-Vector-16-HX-AI-A2XWIG:~/practice$
```

9. install the project (???????)

```
python -m pip install -e source/<given-project-name>
```

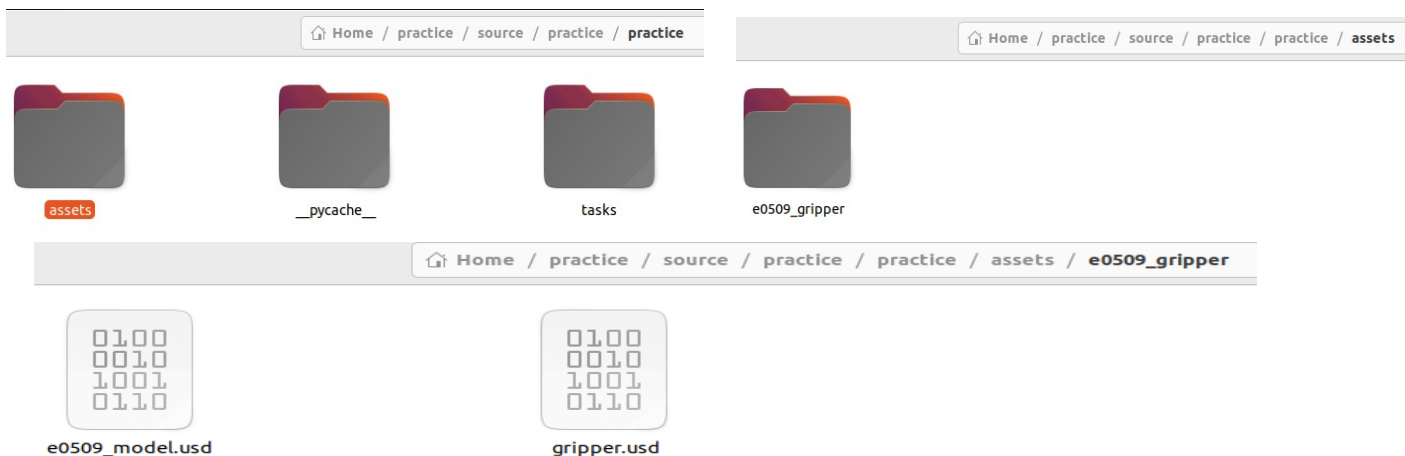
or in my case:

```
python -m pip install -e source/practice
```

```
(env_isaacclab) user@user-Vector-16-HX-AI-A2XWIG:~/practice$ python -m pip install -e source/practice
Obtaining file:///home/user/practice/source/practice
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: psutil in /home/user/miniconda3/envs/env_isaacclab/lib/python3.11/site-packages (from practice==0.1.0) (5.9.8)
Building wheels for collected packages: practice
  Building editable for practice (pyproject.toml) ... done
  Created wheel for practice: filename=practice-0.1.0-0.editable-py3-none-any.whl size=2915 sha256=341e29aaa9b347b146b9436634d3bb5c85268a332459d61bf3da04baf38bc4af
  Stored in directory: /tmp/pip-ephem-wheel-cache-k779z10g/wheels/af/f0/36/9a1d45814eaeda261fe28e4958522e423a7e375e14592e4c5d
Successfully built practice
Installing collected packages: practice
  Attempting uninstall: practice
    Found existing installation: practice 0.1.0
    Uninstalling practice-0.1.0:
      Successfully uninstalled practice-0.1.0
  Successfully installed practice-0.1.0
```

10. create an 'assets' file within this locations:

```
/home/user/practice/source/practice/practice
```



11. open the cfg.py file in this location:

/home/user/practice/source/practice/practice/tasks/direct/practice/practice_env_cfg.py

and replace the ENTIRE file with the following:

```
# Copyright (c) 2022-2025, The Isaac Lab Project Developers
# (https://github.com/isaac-sim/IsaacLab/blob/main/CONTRIBUTORS.md).
# All rights reserved.
#
# SPDX-License-Identifier: BSD-3-Clause

from pathlib import Path

_PKG_ROOT = Path(__file__).resolve().parents[3] # .../source/practice/practice
DOOSAN_USD = str(_PKG_ROOT / "assets/e0509_gripper/e0509_model.usd")

import isaacsim as sim_utils
from isaacsim.actuators import ImplicitActuatorCfg

from isaacsim.assets import ArticulationCfg
from isaacsim.envs import DirectRLEnvCfg
from isaacsim.scene import InteractiveSceneCfg
from isaacsim.sim import SimulationCfg
from isaacsim.utils import configclass

@configclass
class PracticeEnvCfg(DirectRLEnvCfg):
    # env
    decimation = 2
    episode_length_s = 5.0
    # - spaces definition
    action_space = 6
    observation_space = 18
    state_space = 0

    # simulation
    sim: SimulationCfg = SimulationCfg(dt=1 / 120, render_interval=decimation)

    # robot(s)
    robot_cfg: ArticulationCfg = ArticulationCfg(
        prim_path="/World/envs/env_*/Robot",
        spawn=sim_utils.UsdFileCfg(
            usd_path=DOOSAN_USD,
            activate_contact_sensors=False,
            articulation_props=sim_utils.ArticulationRootPropertiesCfg(
                enabled_self_collisions=False,
                solver_position_iteration_count=8,
                solver_velocity_iteration_count=1,
```

```

    ),
    collision_props=sim_utils.CollisionPropertiesCfg(
        collision_enabled=True,
        contact_offset=0.005,
        rest_offset=0.0,
    ),
),
init_state=ArticulationCfg.InitialStateCfg(
    pos=(-0.45, 0.0, 0.53),
    joint_pos={
        "joint_1": 0.0,
        "joint_2": -1.2,
        "joint_3": 1.5,
        "joint_4": -1.8,
        "joint_5": 0.0,
        "joint_6": 1.6,
        "rh_*": 0.0,
    },
),
actuators={
    "arm": ImplicitActuatorCfg(
        joint_names_expr=["joint_[1-6]"],
        stiffness=1500.0,
        damping=100.0,
    ),
    "gripper": ImplicitActuatorCfg(
        joint_names_expr=["rh_l1", "rh_r1_joint"],
        stiffness=2e4,
        damping=1e2,
    ),
},
)

```

```

# scene
scene: InteractiveSceneCfg = InteractiveSceneCfg(num_envs=64, env_spacing=4.0,
replicate_physics=True)

```

```

# custom parameters/scales
# --- Reach task params ---
action_scale = 0.25      # rad per step for joint delta
target_radius = 0.05     # success threshold [m]
target_pos = (0.45, 0.0, 0.55) # start fixed target (we randomize later)
rew_scale_dist = 1.0
rew_scale_action = 0.01
# - controllable joint
# - reward scales
rew_scale_terminated = -2.0
rew_scale_pole_pos = -1.0
rew_scale_cart_vel = -0.01
rew_scale_pole_vel = -0.005
# - reset states/conditions

```

12. open the file 'practice_env.py' located in:
/home/user/practice/source/practice/practice/tasks/direct/practice

and replace the ENTIRE file contents with this:

```
# Copyright (c) 2022-2025, The Isaac Lab Project Developers
# (https://github.com/isaac-sim/IsaacLab/blob/main/CONTRIBUTORS.md).
# All rights reserved.
#
# SPDX-License-Identifier: BSD-3-Clause

from __future__ import annotations

import torch
from collections.abc import Sequence

import isaacsim as sim_utils
from isaacsim.assets import Articulation
from isaacsim.envs import DirectRLEnv
from isaacsim.sim.spawners.from_files import GroundPlaneCfg, spawn_ground_plane

from .practice_env_cfg import PracticeEnvCfg

class PracticeEnv(DirectRLEnv):
    cfg: PracticeEnvCfg

    def __init__(self, cfg: PracticeEnvCfg, render_mode: str | None = None, **kwargs):
        super().__init__(cfg, render_mode, **kwargs)

        # ---- Arm joint mapping (known joint names) ----
        self._arm_joint_names = [f"joint_{i}" for i in range(1, 7)]
        self._arm_dof_ids, _ = self.robot.find_joints(self._arm_joint_names)

        # ---- Pick an end-effector body ONCE ----
        candidates = ["tool0", "ee_link", "end_effector", "flange", "tcp"]
        ee_id = None
        for name in candidates:
            try:
                ids, _ = self.robot.find_bodies(name)
                if len(ids) > 0:
                    ee_id = int(ids[0])
                    break
            except Exception:
                pass

        # Fallback: last body
        self._ee_body_id = ee_id if ee_id is not None else -1

        # ---- Fixed target position in "env-local" frame ----
        # We convert it to world frame by adding scene.env_origins each step.
```

```

    self.target_pos = torch.tensor(self.cfg.target_pos,
device=self.device).repeat(self.num_envs, 1)

    # Optional: helpful debug (uncomment once if you want)
    # print("EE body id:", self._ee_body_id)
    # try:
    #     print("EE body name:", self.robot.body_names[self._ee_body_id])
    # except Exception:
    #     pass

def _setup_scene(self):
    # Create robot articulation from cfg
    self.robot = Articulation(self.cfg.robot_cfg)

    # Add ground plane
    spawn_ground_plane(prim_path="/World/ground", cfg=GroundPlaneCfg())

    # Clone and replicate environments
    self.scene.clone_environments(copy_from_source=False)

    # Explicitly filter collisions for CPU simulation
    if self.device == "cpu":
        self.scene.filter_collisions(global_prim_paths=[])

    # Add articulation to scene
    self.scene.articulations["robot"] = self.robot

    # Add lights
    light_cfg = sim_utils.DomeLightCfg(intensity=2000.0, color=(0.75, 0.75, 0.75))
    light_cfg.func("/World/Light", light_cfg)

def _pre_physics_step(self, actions: torch.Tensor) -> None:
    # Clamp actions to [-1, 1]
    self.actions = torch.clamp(actions, -1.0, 1.0)

def _apply_action(self) -> None:
    # Delta joint-position targets for the 6 arm joints
    q_arm = self.robot.data.joint_pos[:, self._arm_dof_ids] # (num_envs, 6)
    dq = self.cfg.action_scale * self.actions # (num_envs, 6)
    q_target = q_arm + dq

    # Apply joint position targets only to arm joints
    self.robot.set_joint_position_target(q_target, joint_ids=self._arm_dof_ids)

def _get_observations(self) -> dict:
    # Arm joint state
    q = self.robot.data.joint_pos[:, self._arm_dof_ids] # (num_envs, 6)
    qd = self.robot.data.joint_vel[:, self._arm_dof_ids] # (num_envs, 6)

    # End-effector world position
    ee_pos = self.robot.data.body_pos_w[:, self._ee_body_id, :] # (num_envs, 3)

```

```

# Target world position (env origin + local target)
target_pos_w = self.scene.env_origins + self.target_pos    # (num_envs, 3)

# Observation = 18 dims
obs = torch.cat([q, qd, ee_pos, target_pos_w], dim=-1)    # (num_envs, 18)
return {"policy": obs}

def _get_rewards(self) -> torch.Tensor:
    ee_pos = self.robot.data.body_pos_w[:, self._ee_body_id, :]
    target_pos_w = self.scene.env_origins + self.target_pos
    dist = torch.linalg.norm(ee_pos - target_pos_w, dim=-1)

    # Dense reward: closer is better
    rew_dist = -self.cfg.rew_scale_dist * dist

    # Small action penalty
    rew_act = -self.cfg.rew_scale_action * torch.sum(self.actions**2, dim=-1)

    return rew_dist + rew_act

def _get_dones(self) -> tuple[torch.Tensor, torch.Tensor]:
    ee_pos = self.robot.data.body_pos_w[:, self._ee_body_id, :]
    target_pos_w = self.scene.env_origins + self.target_pos
    dist = torch.linalg.norm(ee_pos - target_pos_w, dim=-1)

    terminated = dist < self.cfg.target_radius
    time_out = self.episode_length_buf >= self.max_episode_length - 1
    return terminated, time_out

def _reset_idx(self, env_ids: Sequence[int] | None):
    if env_ids is None:
        env_ids = self.robot._ALL_INDICES
    super()._reset_idx(env_ids)

    # Default joint state from robot init/defaults
    joint_pos = self.robot.data.default_joint_pos[env_ids].clone()
    joint_vel = self.robot.data.default_joint_vel[env_ids].clone()

    # Place root at env origin
    default_root_state = self.robot.data.default_root_state[env_ids].clone()
    default_root_state[:, :3] += self.scene.env_origins[env_ids]

    # Write to sim
    self.robot.write_root_pose_to_sim(default_root_state[:, :7], env_ids)
    self.robot.write_root_velocity_to_sim(default_root_state[:, 7:], env_ids)
    self.robot.write_joint_state_to_sim(joint_pos, joint_vel, None, env_ids)

    # Reset target (fixed for now)
    self.target_pos[env_ids] = torch.tensor(self.cfg.target_pos, device=self.device)

```

13. we have successfully created all the files we need for running the reinforcement learning. Now we can run the reinforcement learning project via this:

```
python scripts/random_agent.py --task Template-Practice-Direct-v0 --num_envs 1
```

```
gymFactory v0.1] 100 times. Consider accessing this interface with carb::getcachedinterface() (Performance warning)
^C(env_isaac) user@user-Vector-16-HX-AI-A2XWIG:~/practice$ python scripts/random_agent.py --task Template-Practice-Direct
-v0 --num envs 1
```

